SDEV 2401

Rapid Backend App Development

Class Based Views - 2

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments

- No Cheating

- Be a good classmate

- Don’t waste your time

- Show up to class

WE ARE ESSENTIAL TO ALBERTA | NAIT

Agenda

On the right is what we will cover today.

- Generic Class-Based Views - CreateView
- UpdateView for Editing Existing Records
- DeleteView for Object Removal
- Mixins for Access Control
- Customizing Class-Based Views with Mixins
- Summary

WE ARE ESSENTIAL TO ALBERTA
NAIT

Generic Class-Based Views - CreateView

Example: Using `CreateView` for New Objects

```
from django.views.generic import CreateView
from myapp.models import Event

class EventCreateView(CreateView):
    model = Event
    fields = ['name', 'date', 'location']
    template_name = 'events/create.html'
    success_url = '/events/'
```

- `CreateView` handles form display and object creation, reducing repetitive code.
- The `fields` attribute specifies which model fields to expose.
- On successful creation, redirects to the given `success_url`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

UpdateView for Editing Existing Records

Example: Editing a Blog Post

```
from django.views.generic import UpdateView
from myapp.models import BlogPost

class BlogPostUpdateView(UpdateView):
    model = BlogPost
    fields = ['title', 'content']
    template_name = 'blog/edit.html'
    success_url = '/blog/'
```

- `UpdateView` provides a form for editing existing objects.
- Automatically loads the object based on URL parameters.
- Handles validation and saving, streamlining update logic.

WE ARE ESSENTIAL TO ALBERTA | NAIT

DeleteView for Object Removal

Example: Deleting a User Profile

```
from django.views.generic import DeleteView
from myapp.models import UserProfile

class UserProfileDeleteView(DeleteView):
    model = UserProfile
    template_name = 'profiles/confirm_delete.html'
    success_url = '/profiles/'
```

- `DeleteView` shows a confirmation page and deletes the object upon confirmation.
- Uses `success_url` to redirect after deletion.
- Helps prevent accidental deletions with built-in confirmation.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Mixins for Access Control

Example: `PermissionRequiredMixin` for Fine-Grained Permissions

```
from django.contrib.auth.mixins import PermissionRequiredMixin
from django.views.generic import ListView
from myapp.models import SecretDocument

class SecretDocumentListView(PermissionRequiredMixin, ListView):
    model = SecretDocument
    permission_required = 'myapp.view_secretdocument'
    template_name = 'documents/secret_list.html'
```

- `PermissionRequiredMixin` restricts access based on user permissions.
- Easily integrates with Django's permission system.
- Mixins can be combined for layered access control.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Customizing Class-Based Views with Mixins

Example: Adding a Timestamp Mixin

```
from django.utils import timezone

class TimestampMixin:
    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['timestamp'] = timezone.now()
        return context

from django.views.generic import TemplateView

class HomeView(TimestampMixin, TemplateView):
    template_name = 'home.html'
```

- Custom mixins can inject reusable logic into views.
- Here, `TimestampMixin` adds the current time to the template context.
- Promotes DRY principles and modular code.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Summary

- Class-Based Views (CBVs) provide a structured way to handle requests using classes.
- Django offers generic CBVs like `CreateView`, `UpdateView`, and `DeleteView` to simplify common patterns.
- Mixins allow for reusable functionality and access control across multiple views.
- These are debated features in the django community, but the intent is to provide a more organized and reusable way to define views, especially as applications grow in complexity.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Example

Let’s go do an example together